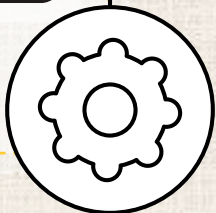


09 강

리스트

첨단공학부
정재화 교수





1. 리스트의 이해

- ✓ 컬렉션의 정의
- ✓ 리스트 생성 및 접근

2. 리스트의 활용

- ✓ 리스트 슬라이싱
- ✓ 리스트 컴프리헨션



학습하기

1

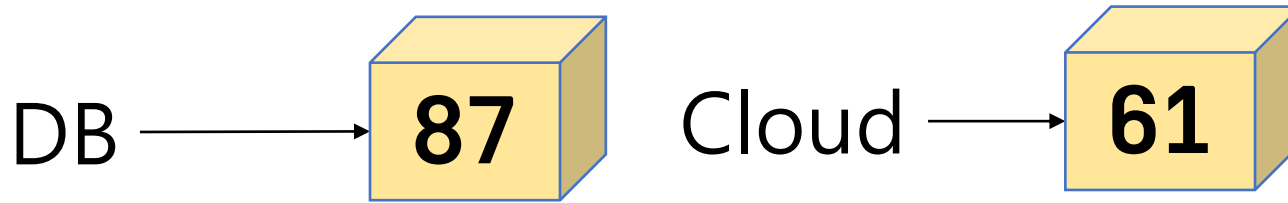
리스트의 이해

리스트(list)

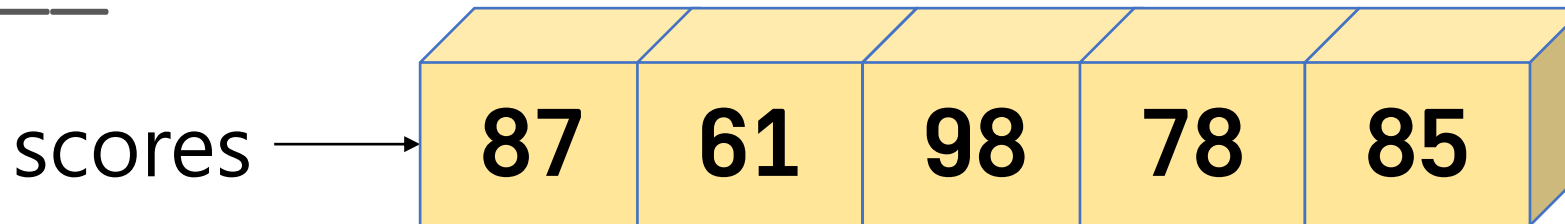
데이터를 순서있게 저장하는 가변(mutable) 시퀀스

✓ 서로 다른 데이터 타입을 함께 저장할 수 있는 구조

■ 변수



■ 리스트



✓ 원소를 추가·삭제·수정할 수 있는 설계

컬렉션의 정의

여러 데이터를 묶어 하나의 구조로 다루기 위해 제공되는 자료 구조의 총칭

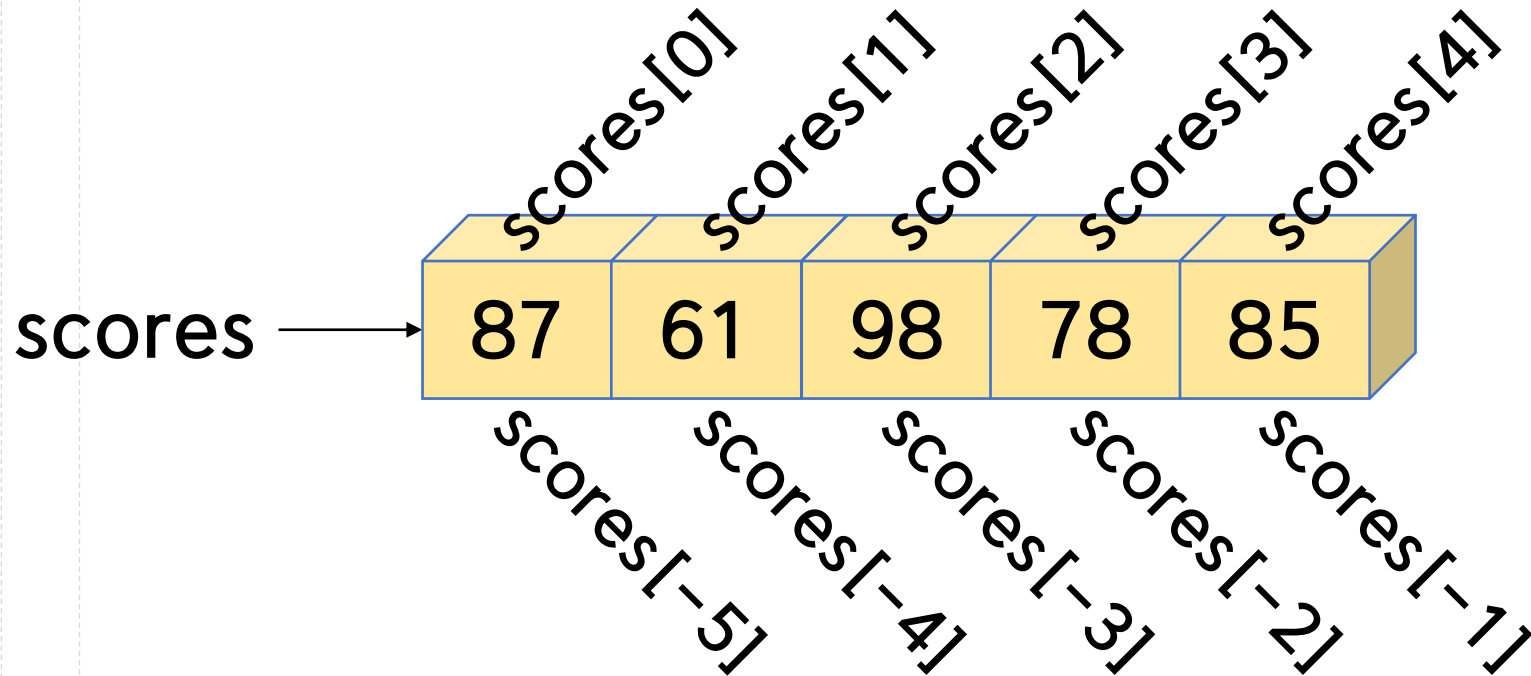
- ✓ 데이터 저장 방식과 접근 방식에 따라
시퀀스·비시퀀스·매핑으로 구분
 - 시퀀스: 리스트(list), 튜플(tuple)
 - 비시퀀스: 세트(set)
 - 매핑: 딕셔너리(dictionary)

데이터 분석, 네트워크 통신, 파일 처리 등 다양한 분야 및 프로그래밍 영역의 기본 도구로 활용

리스트 원소의 접근

접근 연산자([,])와 인덱스 번호 사용

- ✓ 양수는 왼쪽부터, 음수는 오른쪽부터 순차 접근



```
print(scores[4])
```

```
print(scores[-3])
```

리스트 생성

구문형식

```
list([원소 시퀀스])  
[원소 시퀀스]
```

```
list1 = list()
```

```
list1s = []
```

```
scores = [87, 61, 98, 78, 85]
```

```
list2 = list(range(3, 6))
```

```
body = [181, 78, "dark brown", "black", True]
```

리스트 관련 연산자

연산자	사용 예	의미
<code>in</code>	<code>x in s</code>	원소 <code>x</code> 가 순서 <code>s</code> 에 포함 여부
<code>not in</code>	<code>x not in s</code>	원소 <code>x</code> 가 순서 <code>s</code> 에 불포함 여부
<code>+</code>	<code>s1 + s2</code>	2개 순서 <code>s1</code> 과 <code>s2</code> 를 연결
<code>*</code>	<code>s * n</code>	순서 <code>s</code> 를 <code>n</code> 번 복사하여 연결
<code><</code> , <code>></code> , <code>=</code> , <code><=</code> , <code>>=</code> , <code>!=</code>	<code>s 연산자 t</code>	<code>s</code> 와 <code>t</code> 의 사전식 비교

`sco1` → 87 61 98 78 85

`sco2` → 87 61 99

`78 in sco1`

`sco1 + sco2`

`sco2 * 3`

`sco1 < sco2`

리스트 관련 내장 함수

연산자	사용 예	의미
<code>len()</code>	<code>len(s)</code>	s의 길이, 즉 s의 원소 개수
<code>min()</code>	<code>min(s)</code>	s에서 가장 작은 원소
<code>max()</code>	<code>max(s)</code>	s에서 가장 큰 원소
<code>sum()</code>	<code>sum(s)</code>	s의 모든 원소의 합

sco1 → 87 61 98 78 85

sco2 → 87 61 99

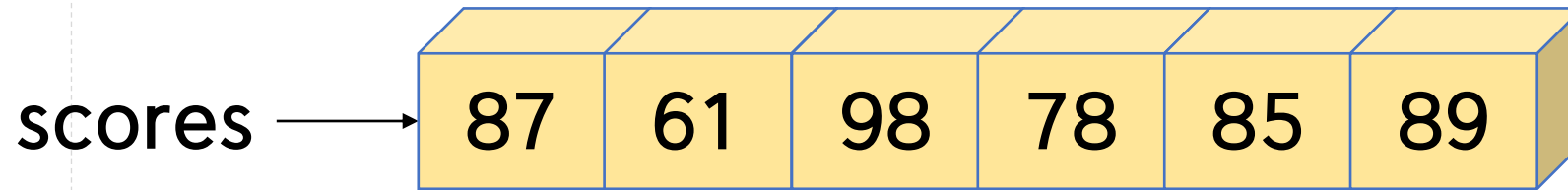
`len(sco1)`

`max(sco1)`

`sum(sco1) < sum(sco2)`

리스트 메소드의 사용

- 리스트 객체에 데이터를 추가, 삭제, 변경, 정렬하는 등의 다양한 작업을 수행



scores.append(89)

객체의 이름

메소드 이름

입력값(파라미터)

리스트 주요 메소드

메소드	의미
<code>append(x: object): None</code>	원소 x를 리스트의 끝에 추가
<code>count(x: object): int</code>	리스트 내, 원소 x의 출현 횟수를 반환
<code>index(x: object): int</code>	리스트에서 원소 x가 첫번째로 나온 위치의 인덱스를 반환
<code>pop(i): object</code>	주어진 위치의 인덱스의 원소를 반환하고 제거
<code>insert(index: int, x: object): None</code>	원소 x를 주어진 인덱스(위치)에 삽입
<code>remove(x: object): None</code>	리스트에서 원소 x를 제거
<code>reverse(): None</code>	리스트 내의 원소들을 역순으로 생성
<code>sort(): None</code>	리스트 내의 원소들을 오름차순으로 정렬

I 유효 평균값과 중앙값 구하기

- 점수 리스트 [9, 8, 10, 7, 9, 8, 10, 6, 9, 8, 7, 9, 10, 8, 9, 5, 8, 9, 7]에서 가장 높은 점수와 가장 낮은 점수를 하나씩 제외하고 나머지 점수의 평균(유효 평균)과 정렬된 점수 중 정확히 한가운데 있는 점수(중앙값)을 출력하시오.

유효 평균값은 8.29입니다
중앙값은 8입니다

학습하기

2

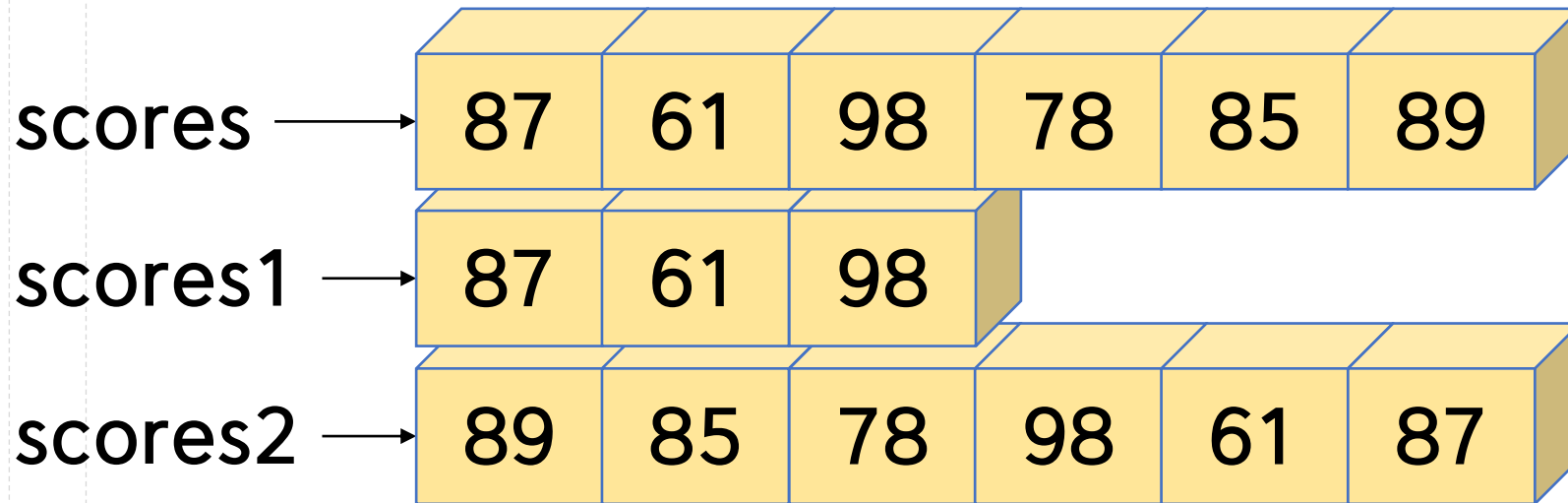
리스트의 활용

리스트 슬라이싱

대상 리스트의 부분 리스트를 생성

구문형식

리스트이름[시작인덱스:끝인덱스:간격]



scores1 = scores[0 : 3] scores[: 4]

scores[3 :] scores2 = scores[:: -1]

리스트 컴프리헨션의 개념

리스트를 간결하게 생성·조작하기 위한 형식

구문형식

[표현식 for 변수 in 리스트]

구문형식

[표현식 for 변수 in 리스트 if 조건]

특징

- ✓ 코드가 짧아지고 가독성 향상
- ✓ 필터링으로 필요 데이터만 추출 가능

리스트 컴프리헨션의 적용 1

원소 수정

```
scoresNbonus = []  
for p in scores:  
    scoresNbonus.append(p+5)
```



```
scoresNbonus = [ p+5 for p in scores]
```

리스트 컴프리헨션의 적용 1

원소 수정

```
scoresNbonus = []  
for p in scores:  
    scoresNbonus.append(p+5)
```



```
scoresNbonus = [p+5 for p in scores]
```

리스트 컴프리헨션의 적용 1

원소 수정

```
scoresNbonus = []  
for p in scores:  
    scoresNbonus.append(p+5)
```



```
scoresNbonus = [p+5 for p in scores]
```

리스트 컴프리헨션의 적용 2

조건 제시

```
scoresNbonus = []  
for p in scores:  
    if p + 5 <= 100:  
        scoresNbonus.append(p+5)
```



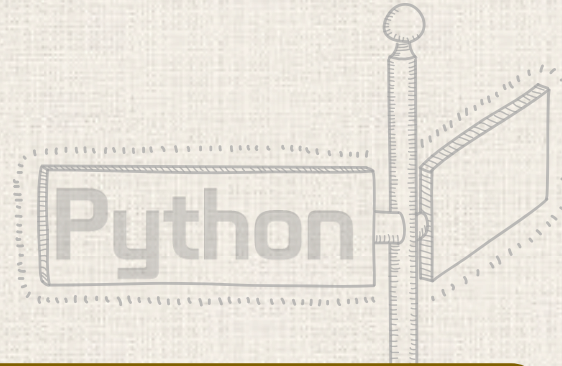
```
scoresNbonus = [ p+5 for p in scores  
                 if p+5 <= 100 ]
```

주식 3일 이동 평균값과 조건 만족 시작일

- 어떤 주식 종목의 최근 30영업일 종가(원) 리스트에서 주가의 단기 흐름을 보기 위해 연속 3일 종가의 평균(3일 이동평균)을 계산하고, 그 값이 11,000원보다 큰 시작일 인덱스와 값을 출력하시오.

시작일	= 9	, 이동평균값	= 11453.33
시작일	= 11	, 이동평균값	= 11233.33
시작일	= 17	, 이동평균값	= 11543.33
시작일	= 18	, 이동평균값	= 11136.67
시작일	= 19	, 이동평균값	= 11790.00
시작일	= 25	, 이동평균값	= 11116.67
시작일	= 27	, 이동평균값	= 11783.33

다음시간에는



9강. 리스트

➡ 10강. 튜플, 세트, 딕셔너리

11강. 다차원 구조